

C Secure Coding Lecture

CVE-2026-31431 — “Copy Fail”

Linux Kernel LPE Vulnerability

Secure Coding Rewrites and Fuzzing Strategies That Would Have Prevented It.

Outline what the lecture is about.

This lecture treats CVE-2026-31431 not as a war story but as a teaching case. The CVE is preserved in full technical detail and used as a vehicle to teach the secure-coding patterns the bug violates and testing strategies that would have caught it. The chapters are as follows:

- Vulnerability overview, at-a-glance facts, and CVSS context.
- Fact-check and errata against the source post (including CVE numbers, dates, and kernel ranges).
- Background: page cache, scatterlists, splice(), AF_ALG and AEAD, the authencsn template — the five subsystems that intersect.
- Anatomy of the vulnerability: the 2017 in-place optimisation, why only authencsn triggers it, the four-byte write, why it is not an out-of-bounds bug.
- The exploit: the write4 primitive, the setuid path, the /etc/passwd path, and why chmod 4711 does not help.
- Secure-coding lecture, part 1 — eight principles violated by Copy Fail, each illustrated with a code example.
- Secure-coding lecture, part 2 — a line-by-line rewrite of algif_aead._aead_recvmmsg() and crypto_authenc_esn_decrypt() showing the secure version side-by-side with the vulnerable original.
- Would fuzzing have caught Copy Fail? An honest analysis — including why eight years of syzkaller did not find it, and the kinds of fuzzing that would have.
- Setting up a fuzzing lab to find Copy-Fail-class bugs: QEMU, KASAN, syzkaller, and a custom page-cache integrity oracle.
- Three reusable harnesses: a syzkaller pseudo-syscall description, a Python property-based harness, and an eBPF runtime invariant detector.
- Static analysis: CodeQL and Smatch rules that would have flagged this composition during code review.
- Operational mitigations for defenders.
- Lessons and takeaways for kernel and systems developers.
- References.

Chapter 1 — Vulnerability At a Glance

Before diving into mechanics, here are the load-bearing facts you will need to keep in mind across the lecture.

CVE	CVE-2026-31431
Public name	Copy Fail
Class	Local privilege escalation (LPE) via controlled 4-byte write into the page cache
Affected component	Linux kernel: crypto/algif_aead.c (AF_ALG AEAD socket interface) interacting with crypto/authencesn.c
Vulnerability introduced	2017 — kernel commit 72548b093ee3 (in-place AEAD optimisation in algif_aead)
Pre-conditions are also rooted in	2011 — commit a5079d084f8b (authencesn added, uses dst as scratch space) and 2015 — commit 104880a6b470 (authencesn ported to the new AEAD interface, introducing the assoclen + cryptlen offset)
Patched by	Mainline 7.0 commit a664bf3d603d; stable 6.19.12 (ce42ee423e58); stable 6.18.22 (fafa0fa2995a)
Affected kernels	4.14 through pre-patch 7.0-rc; 6.18.x < 6.18.22; 6.19.x < 6.19.12 — wherever CONFIG_CRYPTO_USER_API_AEAD is enabled (default on every mainstream distribution)
Disclosure date	April 29, 2026 (Theori / Xint Code research team — researcher Taeyang Lee, AI-assisted via Xint Code)
Public exploit size	732-byte Python script (Theori PoC); also C and Go ports
CVSS v3.1	7.8 (High) — Local / Low / None / Unchanged / High-High-High
Container blast radius	Crosses container boundaries; page cache is per-filesystem inode, not per-namespace; reachable from RuntimeDefault seccomp profiles in Kubernetes
Forensic visibility	Invisible to on-disk integrity tools (Tripwire, AIDE, IMA, container image scanners) — disk file is never modified.

The one sentence to remember

Copy Fail is a logic flaw in which a 2017 “in-place” optimisation in the AF_ALG AEAD socket interface let page-cache pages of a readable file end up in the kernel’s writable scatterlist. The authencesn IPsec-ESN template — never intended for userspace exposure — uses that scatterlist as scratch space, producing a deterministic four-byte write into the page cache of any file the unprivileged caller can open.

Chapter 2 — Fact-Check and Errata

Before the technical content, we anchor a small set of facts that are misstated or imprecise in the source blog. Citations to primary sources (kernel.org, Theori/Xint, oss-security, distribution advisories) are in the References chapter.

2.1 Dirty COW CVE is 2016-5195, not 2016-5341

r3tr074, “CVE-2026-31431: Copy Fail” <https://retr0.zip/blog/cve-2026-31431-copy-fail.html> the source blog to this lecture refers to Dirty COW as CVE-2016-5341. The correct identifier is CVE-2016-5195. This is verified against MITRE, NVD, Red Hat, Wikipedia, and the original disclosure site dirtycow.ninja. The correct identifier is used throughout this lecture.

2.2 Disclosure timeline

The Linux CNA assigned and tracked CVE-2026-31431 starting April 22, 2026; the public coordinated disclosure was April 29, 2026, by Theori, with patches landing in mainline (7.0) on April 1, 2026 — pre-disclosure to give distributions a head start. Most distributions shipped patched kernels between April 29 and May 1, 2026.

2.3 Affected kernel range

The Openwall oss-security advisory and the Linux CNA both report that the issue was introduced in 4.14 with commit 72548b093ee3 and fixed in 6.18.22 (fafa0fa2995a), 6.19.12 (ce42ee423e58), and 7.0 (a664bf3d603d). Any kernel between those without the backport is vulnerable if CONFIG_CRYPT_USER_API_AEAD is enabled (the default on every mainstream distribution).

2.4 Constants and syscall names

AF_ALG = 38 (include/linux/socket.h), SOL_ALG = 279 (include/linux/socket.h), and the ALG_SET_* option numbers (KEY=1, IV=2, OP=3, AEAD_ASSOCLEN=4, AEAD_AUTHSIZE=5) are confirmed in include/uapi/linux/if_alg.h. The source blog cites these correctly.

2.5 Page-cache-corruption framing

The source draws a parallel between Copy Fail and Dirty COW. Both end with an in-memory page-cache modification while the on-disk file remains untouched, but the mechanisms differ: Dirty COW won a race against the COW fault handler; Copy Fail is a deterministic logic flaw that routes a misaddressed write to a crypto subsystem page via splice() and an SGL chain. Calling both “page-cache corruption” is descriptively correct; Copy Fail is distinguished by being race-free, deterministic, and cross-architecture portable.

2.6 “Crypto subsystem has no concept of page ownership”

The source frames the failure as “the crypto subsystem has no concept of page ownership.” That is descriptively correct, but it locates the fault one layer too low. The crypto core trusts its callers to supply appropriate scatterlists. The actual fault is in `algif_aead` — the `AF_ALG` adapter — which broke the invariant that `req->dst` points only to kernel-private, writable buffers. Throughout this lecture, we name the layer at fault more precisely, because secure-coding fixes apply at that layer.

2.7 Container claim — accurate, with a nuance

The source notes that the bug crosses container boundaries because the page cache is per-filesystem. Confirmed: independent Kubernetes reproductions (Juliet Security, Microsoft Security) show that pods running under `RuntimeDefault seccomp` and `Pod Security Standards “Restricted”` can still open `AF_ALG` sockets and reach the vulnerable path. The default Docker / `containerd` profile denies `AF_VSOCK` (40) but not `AF_ALG` (38). The exploit’s effect on the host is, however, transient — a host reboot drops the corrupted page from cache.

Chapter 3 — The Five Kernel Subsystems That Intersect

Copy Fail spans five kernel subsystems that rarely appear together in any single review. The lecture-style approach is to introduce each in isolation, then show how their composition creates the bug.

3.1 The Page Cache

When userspace calls `read()` on a regular file, the kernel does not normally fetch data from disk. It first looks in the page cache — a system-wide, in-memory cache of file data indexed by (inode, offset). If the page is present from a previous read or execution by any process, the kernel returns the cached page. Otherwise, it reads from disk, caches it, and returns it.

Three properties matter for this bug:

- Per-filesystem, not per-namespace. Two processes in different containers, cgroups, mount namespaces, or user namespaces that read the same file on the same filesystem will see the same physical pages.
- Same page back execution. `execve()` loads a program image out of the page cache, not freshly from disk. Therefore, corrupting the cached image corrupts the running program.
- Dirty-bit hygiene. Normal write paths (`write()` and `mmap` stores) mark the page dirty, and the kernel will eventually write it back to disk. Copy Fail's four-byte write does not touch that path: the dirty bit stays clear, the file on disk is unchanged, but every reader of the file from cache sees the corrupted bytes until the page is evicted (or the system reboots).

Why this is a forensic blind spot

On-disk integrity tooling (AIDE, Tripwire, IMA file hashes, container image scanners, supply-chain artefact verification) rehashes file bytes from disk and finds no anomaly. Detection has to be runtime-behavioural (eBPF, audit) or kernel-internal.

3.2 Scatter-Gather Lists

Kernel buffers are often physically discontinuous. A network packet may span multiple `skb` fragments; a file read may span pages allocated at different times. The kernel describes such buffers as a sequence of (page, offset, length) entries — a scatterlist.

A single struct scatterlist (`include/linux/scatterlist.h`) has four relevant fields:

```
struct scatterlist {
    unsigned long    page_link;    /* struct page* with 2 flag bits in the low bits */
    unsigned int     offset;       /* byte offset within the page */
    unsigned int     length;       /* contiguous length covered by this entry */
    dma_addr_t       dma_address; /* physical address for DMA */
};
```

page_link is overloaded. Its low two bits encode flags:

- **Bit 0 — SG_CHAIN:** this entry is not a buffer fragment but a pointer to another array so that arrays can be chained.
- **Bit 1 — SG_END:** this is the terminal entry of the chain.

The crypto subsystem reads and writes data at byte offsets inside (possibly chained) scatterlists via the helper scatterwalk_map_and_copy(). It looks like this in mainline (crypto/scatterwalk.c, abridged):

```
void scatterwalk_map_and_copy(void *buf, struct scatterlist *sg,
                             unsigned int start, unsigned int nbytes, int out)
{
    struct scatter_walk walk;
    struct scatterlist tmp[2];

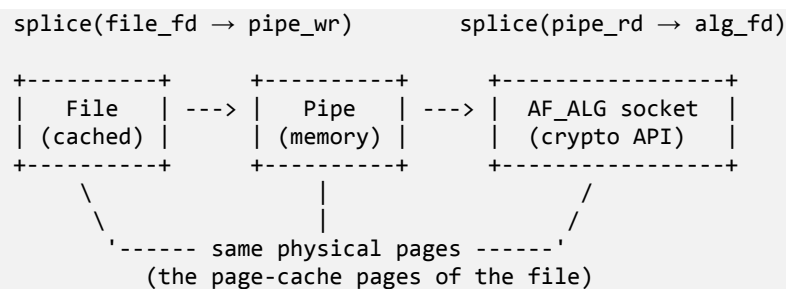
    if (!nbytes) return;
    sg = scatterwalk_ffwd(tmp, sg, start);    /* skip ahead by 'start' */
    scatterwalk_start(&walk, sg);
    scatterwalk_copychunks(buf, &walk, nbytes, out); /* out=1 means write */
    scatterwalk_done(&walk, out, 0);
}
```

This helper performs the four-byte write that defines the Copy Fail primitive. It validates the chain length and offset; it does not validate the provenance of the page it lands on. That is the missing invariant.

3.3 splice() and Zero-Copy I/O

splice() moves data between a file descriptor and a pipe without copying through userspace. Instead of read()-ing bytes into a userland buffer and write()-ing them out again, splice() transfers page references. A pipe created with splice() does not hold a copy of the file data; it holds pointers to the same physical pages already in the page cache.

When the data is then spliced out of the pipe into another file descriptor (in our case, an AF_ALG socket), the destination's internal buffer also receives those page references. Net effect: the kernel's crypto subsystem ends up with direct pointers to page-cache pages of whatever file was spliced, and unless the crypto adapter copies them, they remain shared.



3.4 AF_ALG and AEAD Decryption

AF_ALG (address family 38) is a Linux socket family that exposes in-kernel cryptographic primitives to userspace. No capability is needed; the `algif_aead` module is auto-loaded on first use. A normal setup binds the socket to an algorithm, configures keys and tag size via `setsockopt(SOL_ALG, ...)`, accepts a per-request file descriptor, and then `sendmsg / recvmsg` per operation:

```
int alg_fd = socket(AF_ALG, SOCK_SEQPACKET, 0); /* AF_ALG = 38 */
struct sockaddr_alg sa = {
    .salg_family = AF_ALG,
    .salg_type   = "aead",
    .salg_name   = "authencsn(hmac(sha256),cbc(aes))"
};
bind(alg_fd, (struct sockaddr *)&sa, sizeof(sa));
```

Authenticated Encryption with Associated Data (AEAD) is a family of algorithms providing both confidentiality and integrity. The buffer the kernel sees at decrypt time has three regions:

AAD (auth'd only)	Ciphertext (decrypted+auth'd)	Auth Tag (verified)
----------------------	----------------------------------	------------------------

<-- assoclen --><----- cryptlen -----><----- authsize ----->

3.5 The authencsn Template — A Brief History

`authencsn` is an AEAD template specialised for IPsec ESP with Extended Sequence Numbers (RFC 4303). Its origin and evolution are the first lesson of this lecture, because the bug is the composition of three changes nobody intended to compose.

- 2011 — commit `a5079d084f8b` adds `authencsn` to the kernel. It uses the caller's destination scatterlist as scratch space for ESN byte rearrangement. Harmless at the time: the only caller was the in-kernel XFRM (IPsec) layer, and `dst` is an `skb` the IPsec code controls end-to-end.
- 2015 — AF_ALG gains AEAD support (`algif_aead.c`), and `authencsn` is ported to the new AEAD interface (commit `104880a6b470`). The port introduces the offset `dst[assoclen + cryptlen]` as the write target. AF_ALG is out of place at this time: `req->src` and `req->dst` are separate scatterlists, so the scratch write is still safe.
- 2017 — commit `72548b093ee3` adds an in-place optimisation to `algif_aead` for decrypt: copy AAD and ciphertext from the TX SGL into the RX buffer, chain the tag pages from the TX SGL into the RX SGL via `sg_chain()`, then set `req->src = req->dst`. If the TX SGL came from a `splice()`, those tag pages are in the page cache. They are now in the destination scatterlist. `authencsn`'s 2011-era scratch write is now an unprivileged write to a page-cache page.

The teaching moment

Each of the three commits, in isolation and against the contemporaneous test suite, was correct. The vulnerability lives in their composition. The first task of this lecture's secure-coding chapters is to ask: what invariants would have caught the composition error?

In terms of format, what authenc does inside `crypto_authenc_esn_decrypt()` is rearrange three 4-byte words before HMAC. The wire layout that arrives is:

Reconstructed AAD (12 bytes):

```
[ seqno_hi (4) | seqno_lo (4) | SPI (4) ]
bytes 0..3      bytes 4..7    bytes 8..11
```

Required HMAC input order (per RFC 4303):

```
[ SPI (4) | seqno_lo (4) | seqno_hi (4) | ciphertext ... ]
```

The two layouts differ, so `authencsn` must reshuffle 8 bytes. It does so via three `scatterwalk_map_and_copy()` calls — and step 3 is the one that becomes the Copy Fail write primitive.

```
Step 1: READ  tmp[0..7]  = dst[0..7]                /* save seqno_hi, seqno_lo */
Step 2: WRITE dst[4..7]  = tmp[0..3]  (seqno_hi)      /* move seqno_hi inside RX buffer
*/
Step 3: WRITE dst[assoclen + cryptlen] = tmp[4..7]  (seqno_lo) /* stash seqno_lo past
ciphertext */
```


Chapter 4 — Anatomy of the Vulnerability

With each subsystem introduced, we can now trace the bug end-to-end at the level of actual kernel code.

4.1 The Vulnerable Path Inside `_aead_recvmsg()`

The relevant function is `_aead_recvmsg()` in `crypto/algif_aead.c`. With the 2017 optimisation present, decrypt requests follow this rough sequence:

```
/* crypto/algif_aead.c, pre-patch, _aead_recvmsg() – simplified */
static int _aead_recvmsg(struct sock *sk, struct msghdr *msg, ...)
{
    struct aead_async_req *areq;
    struct scatterlist *rsgl_src;
    /* ... lock the socket, count and allocate RX pages ... */

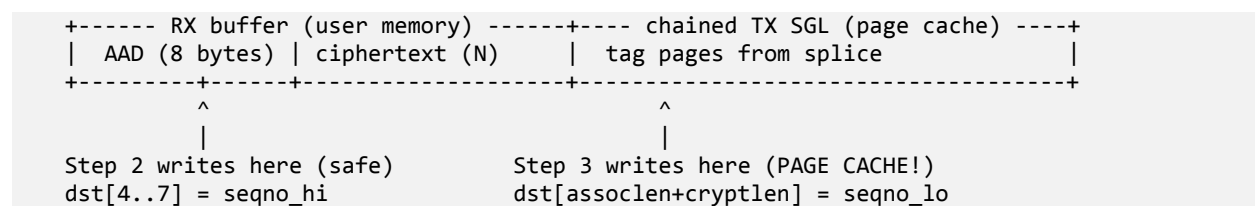
    /* (1) Copy AAD and ciphertext bytes from the TX SGL into the RX buffer. */
    memcpy_sglist(rsgl_src, areq->tsgl, used - authsize);

    /* (2) Chain the LAST authsize bytes of the TX SGL – which, after splice(), are
     *      page-cache pages of the spliced file – onto the end of the RX SGL. */
    sg_chain(rsgl_src, rsgl_count + 1, areq->tsgl_tag);

    /* (3) In-place: both src and dst point at the combined RX+chained SGL. */
    aead_request_set_crypt(&areq->cra_u.aead_req,
                          rsgl_src,          /* src */
                          rsgl_src,          /* dst (== src) */
                          used, ctx->iv);

    /* (4) Run the AEAD. authencsn writes at dst[assoclen + cryptlen]. */
    err = crypto_aead_decrypt(&areq->cra_u.aead_req);
    /* err == -EBADMSG (HMAC fails) – but the 4-byte write has already landed. */
}
```

Visually, after step 2, dst looks like this:



The boundary between “safe to write” (the freshly-allocated RX buffer) and “must not write” (the chained page-cache pages) is just a byte offset inside one logical buffer. Nothing in the type system, the API, or the documentation tells `authencsn` to stop short of that offset.

4.2 Where Inside authencsn the Damage Happens

`crypto_authenc_esn_decrypt()` in `crypto/authencsn.c` implements step 3. Here is the on-tree code, abridged to highlight the issue:

```
/* crypto/authencsn.c - crypto_authenc_esn_decrypt(), abridged */
static int crypto_authenc_esn_decrypt(struct aead_request *req)
{
    /* ... setup, derive assoclen, cryptlen, authsize, dst = req->dst ... */
    u8 *tmp = areq_ctx->tail; /* 8-byte scratch in request ctx */

    /* Move high-order bits of sequence number to the end. */
    scatterwalk_map_and_copy(tmp, src, 0, 8, 0); /* READ */
    if (src == dst) { /* in-place */
        scatterwalk_map_and_copy(tmp, dst, 4, 4, 1); /* WRITE safe */
        scatterwalk_map_and_copy(tmp + 4, dst, assoclen + cryptlen,
                                4, 1); /* WRITE COPY-FAIL */
        dst = scatterwalk_ffwd(areq_ctx->dst, dst, 4);
    } else {
        /* ... separate-buffer path (not exercised when AF_ALG is in-place) ... */
    }
    /* ... compute HMAC, decrypt ciphertext, then decrypt_tail restores ... */
}
```

The last argument 1 to `scatterwalk_map_and_copy` means “write”. The third argument `assoclen + cryptlen` is exactly the offset where, in `algif_aead`’s in-place layout, the chained page-cache pages begin. `authencsn` deposits four attacker-controlled bytes there.

4.3 Why Only authencsn Triggers This

The in-place optimisation has shipped since 2017 and is used by every AEAD algorithm exposed through `AF_ALG`: `authenc`, `gcm`, `ccm`, `chachapoly`, and so on. None of them corrupts the page cache. Tracing the writes for a normal AEAD decrypt:

- **HMAC engine** — reads from `dst` over the AAD and ciphertext to compute the tag, but writes the result into a separate, request-local buffer (`areq_ctx->tail`). Nothing is written back to `dst`.
- **Cipher engine** — writes plaintext into `dst`, strictly within `dst[assoclen .. assoclen + cryptlen - authsize]`. That range lives entirely in the RX buffer copied at step 1.
- **Tag verification** — reads the supplied tag from `req->src` (a read), copies it into a local buffer, compares it against the computed HMAC, then discards.

All three either read from `dst` or write only into the safe RX portion. No general-purpose AEAD pipeline writes into the chained tag-page region during decrypt. `authencsn` is unique because its IPsec-ESN reshuffle is the only code in the kernel that explicitly writes to `dst[assoclen + cryptlen]` during decrypt — and it does so because, in 2011, nobody anticipated that `dst` might span more than one trust domain.

4.4 Why This Is Not an Out-of-Bounds Write

The phrase “writes four bytes past the ciphertext boundary” sounds like a heap overflow. It is not.

The destination scatterlist is the correct length. The entry at offset `assoclen + cryptlen` is a valid struct scatterlist with a valid struct page pointer. `scatterwalk_map_and_copy()` does not exceed any declared range. No bounds check fires; no allocator metadata is touched; no memory tag is wrong.

The bug is one of ownership, not size:

- In the normal IPsec decrypt path, the page at offset `assoclen + cryptlen` is a kernel-allocated skb page used as transient tag scratch space. Writing four bytes there is benign.
- With the in-place optimisation plus `splice()`, the page at the same offset is instead a page cache page that backs an on-disk file readable by the attacker. The four-byte write is architecturally identical — it just lands on a page that should never have been reachable from this scatterlist position.

Why eight years of memory-safety tooling missed this.

Memory sanitisers (KASAN, KFENCE, MTE, sparse, syzkaller harnesses) look for size violations, lifetime errors, and undefined behaviour. They do not model “this page belongs to a different security domain than the rest of the buffer.” Copy Fail — and Dirty COW before it — is a class of bug that those tools are structurally blind to. Chapters 8–10 examine what tooling would have caught it.

4.5 What the Attacker Controls

- **The four bytes were written.** Bytes 4..7 of the supplied AAD become `seqno_lo` and are written verbatim into the page-cache page.
- **The target file.** Any file the attacker can `open()` with `O_RDONLY` and pass to `splice()`.
- **The target byte offset.** A deterministic function of the file offset given to `splice()` and of the `assoclen / cryptlen` pair set per request, allowing single-page writes at arbitrary offsets within the target file’s page cache.

The HMAC check at the end of the AEAD decrypt always fails (the attacker controls neither the key nor the tag), so `recvmsg()` returns `-EBADMSG`. By the time the error returns, however, the four bytes are already on a page-cache page. The HMAC failure is irrelevant to the corruption.

Chapter 5 — Exploiting the Primitive

With the primitive established, exploitation reduces to a one-line API: `write4(target_fd, offset, value_4_bytes)`. It iterates a payload into the in-memory copy of any readable file.

5.1 The `write4` Primitive

Step by step, `write4` does the following:

- Build an 8-byte AAD whose bytes 4..7 are the value to be written. Bytes 0..3 are inert filler — they end up as `tmp[0]` (`seqno_hi`) and are copied to `dst[4..7]` inside the safe RX buffer.
- `sendmsg()` the AAD into the `AF_ALG` request socket with `MSG_MORE`, plus `cmsg` headers setting `ALG_OP=DECRYPT`, `ALG_SET_IV` (16 zero bytes), and `ALG_SET_AEAD ASSOCLen=8`.
- `Splice` (`offset + 4`) bytes of the target file into a pipe, then splice the pipe into the request socket. The crypto subsystem now holds the target file's page-cache pages as TX scatterlist entries.
- `recv()` to trigger the decrypt. `authencsn` shuffles ESN bytes; step 3 of the shuffle writes `seqno_lo` to `dst[assoclen + cryptlen]`, landing on a page-cache page of the target file.
- Ignore the inevitable `-EBADMSG`; the four bytes have already been written.

As C, the primitive is roughly forty lines:

```
void write4(int req_fd, int target_fd, off_t offset, const uint8_t value[4])
{
    uint8_t aad[8] = { 0,0,0,0, value[0], value[1], value[2], value[3] };

    struct msghdr msg = { /* iov = aad, cmsg = decrypt+IV+assoclen=8 */ };
    sendmsg(req_fd, &msg, MSG_MORE);

    int pipefd[2]; pipe(pipefd);
    splice(target_fd, NULL, pipefd[1], NULL, offset + 4, 0);
    splice(pipefd[0], NULL, req_fd, NULL, offset + 4, 0);

    char trash[4096];
    recv(req_fd, trash, sizeof(trash), 0); /* -EBADMSG, ignored */
    close(pipefd[0]); close(pipefd[1]);
}
```

5.2 Classical Path — Corrupt a setuid Binary

Theori's PoC iterates `write4` across `/usr/bin/su`, replacing the cached program image with a small shellcode that calls `setuid(0)` and `execve("/bin/sh")`. A subsequent `execve("/usr/bin/su", ...)` loads the program from the page cache, runs the attacker's shellcode, and yields a root shell. `/usr/bin/su` is chosen because it is `setuid-root`, so `execve()` applies `setuid` elevation when control reaches the rewritten entry point. `/usr/bin/passwd`, `/usr/bin/sudo`, `/usr/bin/mount`, and `/usr/bin/pkexec` are equivalent targets.

5.3 Alternative Path — Rewrite `/etc/passwd`

The source author contributes a strictly broader strategy. Rather than corrupting a `setuid` binary, he overwrites the UID field of the attacker's own line in `/etc/passwd`. The flow is:

- Open `/etc/passwd` `O_RDONLY` and locate the attacker's line, e.g., `alice:x:1000:1000:Alice,...:/home/alice:/bin/bash`.
- Compute the byte offset of the UID field (the third colon-separated field).
- Use `write4` to replace the four ASCII digits "1000" with "0000" in the page cache. The line length is unchanged, so no surrounding bytes are disturbed.
- Run `su - alice` and authenticate with the attacker's own (valid) password — PAM consults `/etc/shadow`, which is untouched.
- After authentication, `su` calls `getpwnam("alice")` to look up the target UID. That lookup reads `/etc/passwd` through the page cache and returns UID 0.
- `su` calls `setuid(0)`. A root shell follows.

Why this matters operationally

The shellcode-into-su approach is convenient but binds the exploit to the presence of a `setuid` binary that the attacker can both open for reading and execute. The `/etc/passwd` approach needs only a world-readable text file (true on essentially every Linux system), the user's own password, and the standard `su` utility. It is a strictly broader primitive — and it invalidates the popular but incomplete `chmod 4711` mitigation discussed below.

5.4 Caveats for the /etc/passwd Path

Practical conditions defenders should know:

- **NSS backends.** Systems whose Name Service Switch (/etc/nsswitch.conf) resolves users through LDAP, AD/SSSD, or systemd-homed will not consult /etc/passwd for the attacker's account; cache poisoning is then moot. Local-only accounts on "files"-only nsswitch remains fully exploitable.
- **Caching daemons.** nscd or sssd can mask the corrupted in-memory state by serving from their own cache. This is not a reliable mitigation, but it reduces success rates.
- **PAM stacks that pin UIDs pre-auth.** The default util-linux su consults getpwnam() after authentication. Hardened PAM modules that pin the UID before authentication may break the exploit; this is rare in practice.
- **Atomic 4-byte writes.** The /etc/passwd PoC only needs one write4 call. Multi-write payloads — e.g., rewriting a binary — require that no page is reclaimed from the cache between writes; otherwise, an earlier write is undone. Attackers anchor pages by holding the file open or by repeatedly reading them.

5.5 Why chmod 4711 Does Not Help

After disclosure, a script circulated that strips world-read permission from common setuid binaries to prevent unprivileged users from open()-ing and splicing them:

```
for b in passwd chsh chfn mount sudo pkexec; do
    p=$(readlink -f "$(command -v "$b")")
    [ -n "$p" ] && [ "$(stat -c %a "$p")" != "4711" ] && chmod 4711 "$p"
done
```

This is incomplete by design. The primitive corrupts any file that the user can read. /etc/passwd is world-readable by necessity (id, ls -l, ps, and every UID-to-name mapping reads it). The chmod script does nothing about /etc/passwd, /etc/group, the dynamic-linker cache, world-readable systemd unit files, and so on. The mitigation provides false confidence.

What actually mitigates the bug

Patch the kernel to include commit a664bf3d603d (or the 6.18.22 / 6.19.12 stable backports);

Block algif_aead module load: echo "install algif_aead /bin/false" >

/etc/modprobe.d/copy-fail.conf and rmmod algif_aead;

Block socket(AF_ALG, ...) via seccomp at the container or systemd-unit level (deny socket() with arg0 == 38);

Or use Cloudflare-style bpf-lsm to deny socket_bind for AF_ALG except for an allow-list of known legitimate consumers.

None of dm-crypt/LUKS, kTLS, IPsec/XFRM, OpenSSL, GnuTLS, or SSH go through AF_ALG; blocking algif_aead is safe for the overwhelming majority of workloads.

Chapter 6 — Secure-Coding Lecture, Part 1: Eight Principles Violated by Copy Fail

This is the heart of the lecture. Copy Fail is not the violation of a single secure-coding rule; it is the simultaneous violation of eight, each of which is independently teachable. For each principle, we state the rule, identify where Copy Fail violates it, and provide a code example that shows both the vulnerable pattern and a secure rewrite.

How to read this chapter

Each principle is presented as: (a) the rule; (b) the violation in Copy Fail; (c) a representative C code example. The example is drawn from kernel-style code, so it is immediately useful for kernel reviewers. Still, the principles generalise to userspace systems programming — any language with raw pointers, shared buffers, or zero-copy I/O.

6.1 Principle: Cross-Subsystem Invariants Must Be Explicit

Two subsystems shared an implicit, undocumented contract:

- `algif_aead` silently assumed that no AEAD algorithm would ever write outside `[assoclen, assoclen + cryptlen - authsize]` of `dst`.
- `authencsn` silently assumed `dst` was a private, kernel-owned scratch buffer end-to-end.

Both assumptions were correct individually; neither was written down. The vulnerability is the gap where the two meet. The rule is: every cross-subsystem invariant must be machine-checkable, or at least documented in a header alongside an assertion that the other side runs.

Vulnerable pattern (implicit contract)

```
/* algif_aead.c – The chaining is done with no contract attached. */
sg_chain(rsgl_src, rsgl_count + 1, areq->tsgl_tag);
aead_request_set_crypt(req, rsgl_src, rsgl_src, used, ctx->iv);

/* authencsn.c – the scratch write assumes dst is private. */
scatterwalk_map_and_copy(tmp + 4, dst, assoclen + cryptlen, 4, 1);
```

Secure rewrite (explicit contract enforced at the boundary)

```
/* include/crypto/aead.h – declare the contract. */
/**
 * aead_request_set_crypt - submit AEAD operation
 * @req: the AEAD request
 * @src: input scatterlist (may contain externally-owned page references)
 * @dst: output scatterlist; MUST point only to kernel-owned, writable
 *       pages and MUST NOT alias @src unless @src is also kernel-owned.
 *       Violations are detected by aead_request_set_crypt() and rejected.
 */
void aead_request_set_crypt(struct aead_request *req,
                           struct scatterlist *src,
                           struct scatterlist *dst,
                           unsigned int cryptlen, u8 *iv);
```

```

/* crypto/aead.c – enforce it. */
void aead_request_set_crypt(...)
{
    if (CRYPTO_DEBUG_PROVENANCE && src == dst) {
        WARN_ON_ONCE(sg_has_borrowed_pages(dst) &&
                    "in-place AEAD on borrowed pages");
    }
    /* ... existing implementation ... */
}

```

The check `sg_has_borrowed_pages()` — defined in §6.2 — exists only because we made ownership explicit. Without explicit ownership, no check is possible.

6.2 Principle: Aliasing Source and Destination Requires an Ownership Proof

In-place operation is a legitimate optimisation when `src` and `dst` are guaranteed to be the same kernel-owned buffer. It is dangerous whenever `src` or `dst` may include pages whose provenance is not kernel-private. The 2017 commit allowed in-place operation without proving that all destination pages were kernel-owned.

Secure rewrite (a typed scatterlist with provenance)

```

/* include/linux/scatterlist_typed.h - new */
enum sg_provenance {
    SG_PROV_KERNEL_OWNED,    /* every page in this list was kmalloc'd by us */
    SG_PROV_BORROWED_RO,     /* contains page-cache or other read-only pages */
    SG_PROV_BORROWED_RW,     /* user-mapped writable pages */
};

struct sg_typed {
    struct scatterlist sgl;
    enum sg_provenance prov;
};

static inline bool sg_typed_is_writable(const struct sg_typed *t)
{
    return t->prov == SG_PROV_KERNEL_OWNED || t->prov == SG_PROV_BORROWED_RW;
}

static inline bool sg_typed_may_alias(const struct sg_typed *a,
                                      const struct sg_typed *b)
{
    /* in-place requires both sides KERNEL_OWNED */
    return a->prov == SG_PROV_KERNEL_OWNED && b->prov == SG_PROV_KERNEL_OWNED;
}

/* crypto/algif_aead.c – propagate provenance into the SGL we build. */
static void rsgl_init_from_splice(struct sg_typed *t, struct page *p) {
    /* Pages from splice() are page-cache refs – borrowed, read-only. */
    t->prov = SG_PROV_BORROWED_RO;
    /* ... fill sgl ... */
}

static int _aead_recvmmsg(...)

```



```
{
    /* Building dst from splice-sourced pages now keeps the borrowed-RO tag. */
    struct sg_typed *dst = build_dst_from_splice(...);
    if (!sg_typed_is_writable(dst))
        return -EPERM; /* refuse instead of corrupting */
    aead_request_set_crypt_typed(req, src, dst, used, iv);
}
```

This is the kernel-C approximation of what a language with linear or affine types (Rust's `&mut`, Ada/SPARK's ownership, ATS) gets for free. In Rust, the borrow checker would have refused to put a `&mut Page` over a buffer that includes immutably borrowed page-cache pages.

6.3 Principle: Never Use Caller-Supplied Buffers as Scratch Space

authencsn used `dst` as a temporary swap area for an 8-byte byte rearrangement — a classic micro-optimisation that saves an 8-byte allocation. The penalty for that saving was eight years of unprivileged page-cache writes. The rule is straightforward: callees use private scratch buffers. They never write speculatively to caller buffers.

Vulnerable pattern (scratch on the caller buffer)

```
/* crypto/authencsn.c – pre-fix sketch */
scatterwalk_map_and_copy(tmp, dst, 0, 8, 0);
scatterwalk_map_and_copy(tmp, dst, 4, 4, 1); /* WRITE */
scatterwalk_map_and_copy(tmp + 4, dst, assoclen + cryptlen, 4, 1); /* WRITE -- COPY FAIL */
/* later, decrypt_tail() reverses these writes */
```

Secure rewrite (private scratch buffer)

```
/* crypto/authencsn.c – keep all rearrangements in a private buffer. */
struct authenc_esn_request_ctx {
    struct scatterlist src_sg[3]; /* dedicated for the rearranged input */
    u8 seq_scratch[8]; /* private scratch, never aliased */
    /* ... existing fields ... */
};

static int crypto_authenc_esn_decrypt(struct aead_request *req)
{
    struct authenc_esn_request_ctx *ctx = aead_request_ctx(req);
    u8 *seq = ctx->seq_scratch;

    /* Pull the 8 ESN bytes into our own buffer (READ only). */
    scatterwalk_map_and_copy(seq, req->src, 0, 8, /* out= */ 0);

    /* Build an alternative input scatterlist that feeds the HMAC the bytes
     * in RFC 4303 order WITHOUT touching dst at all. */
    sg_init_table(ctx->src_sg, 3);
    sg_set_buf(&ctx->src_sg[0], &seq[4], 4); /* seqno_lo */
    sg_set_buf(&ctx->src_sg[1], &seq[0], 4); /* seqno_hi */
    sg_chain(ctx->src_sg, 3,
             scatterwalk_ffwd(ctx->ffwd_sg, req->src, 8)); /* SPI + ciphertext */

    /* Feed ctx->src_sg to the HMAC engine. */
    /* ... rest of decrypt ... */
}
```

This eliminates every write to dst from the reshuffle path. Even if the in-place optimisation were reintroduced and even if dst contained page-cache pages, no write originates here—defence in depth, achieved by deleting a write.

6.4 Principle: Trust-Boundary Hardening (the splice → crypto boundary)

When data crosses a trust boundary into a subsystem that may treat it as writable, the boundary code must establish provenance. `algif_aead` is the boundary between `AF_ALG` (anything userspace sends, including splice page references) and the crypto core. It is the right layer to enforce: “any page that arrived here by reference must be copied before being placed in a writable destination scatterlist.”

Secure rewrite (defensive copy at the boundary)

```
/* crypto/algif_aead.c – the fix that the upstream patch effectively implements. */
static int _aead_recvmmsg(...)
{
    /* Always operate out-of-place. */
    struct scatterlist *src = areq->tsgl;          /* may contain splice page refs */
    struct scatterlist *dst = rsgl_dst;           /* freshly kmalloc'd, ours */

    /* The only thing we copy proactively is the AAD; the ciphertext is read by
     * The cipher is written into dst, never read-modify-written on src. */
    memcpy_sglist(dst, src, areq->assoclen);

    aead_request_set_crypt(&areq->cra_u.aead_req, src, dst, used, ctx->iv);
    err = crypto_aead_decrypt(&areq->cra_u.aead_req);
    /* result is delivered to userspace from dst, then freed. */
}
```

This is essentially what commit `a664bf3d603d` does. The mainline fix message is unceremonious: “There is no benefit in operating in-place in `algif_aead` since the source and destination come from different mappings.” Read another way: the 2017 commit reused the optimisation pattern used by in-kernel callers (`XFRM`, `dm-crypt`) where `src` and `dst` genuinely share mappings; in `AF_ALG`, that precondition was always false.

6.5 Principle: “In-Bounds” Is Not the Same as “Safe”

`KASAN`, `KMSAN`, `KFENCE`, `syzkaller`, `sparse`, `smatch` and `Coverity` all consider this code clean. The write at `scatterwalk_map_and_copy(tmp + 4, dst, assoclen + cryptlen, 4, 1)` is well within the bounds of `dst`, hits a fully mapped page, and uses the right primitive. Yet it is the bug. The rule is: provenance and ownership are first-class safety properties. They must be checked alongside bounds and lifetimes.

Secure rewrite (a debug-only provenance assertion)

```
/* crypto/scatterwalk.c – a defence-in-depth runtime check. */
#ifdef CONFIG_DEBUG_CRYPTOPROVENANCE
static void wcheck_page(struct scatter_walk *w)
{

```

```

    struct page *p = scatterwalk_page(w);
    if (!p) return;
    /* If the page is part of the page cache of a regular file, a kernel
     * Write through this path is almost certainly a bug. */
    if (p->mapping && p->mapping->host &&
        S_ISREG(p->mapping->host->i_mode)) {
        WARN_ONCE(1,
            "scatterwalk: write to page-cache page of inode %lu\n",
            p->mapping->host->i_ino);
    }
}
#else
static inline void wcheck_page(struct scatter_walk *w) {}
#endif

void scatterwalk_map_and_copy(void *buf, struct scatterlist *sg,
                             unsigned int start, unsigned int nbytes, int out)
{
    /* ... existing setup ... */
    if (out)
        wcheck_page(&walk); /* fire WARN on write to page-cache page */
    /* ... existing copy ... */
}

```

In a kernel built with `CONFIG_DEBUG_CRYPTO_PROVENANCE=y`, the Copy Fail primitive would have produced a WARN on the very first write, with a stack trace pointing directly at `authencesn`. Such an assertion would have caught the bug on day one of the 2017 commit's testing — if the configuration option had existed.

6.6 Principle: Performance Optimisations Affecting Object Provenance Need Cross-Subsystem Review

The 2017 commit was a local optimisation in `algif_aead`. The reviewer audience was “people who care about `algif_aead`.” But the commit changed the provenance of pages flowing into `req->dst`, which is an invariant that other subsystems (every AEAD implementation in `crypto/`) relied on. A change with that reach requires reviewers from `crypto/`, `mm/` (for splice page-cache semantics), and ideally `net/xfrm/` (because IPsec authors maintain `authencesn`).

A practical operational rule: any patch that introduces `sg_chain()`, changes `req->src == req->dst`, or touches splice page handoff should attract reviewers from `crypto`, `mm`, and `net/xfrm`. This is a process, not code, but it is the single highest-leverage change the kernel community could make in response to Copy Fail.

Code-level analogue (an in-tree CODEOWNERS-style hint)

```

/* MAINTAINERS file additions */
CRYPTO API
M:   Herbert Xu <herbert@gondor.apana.org.au>
R:   crypto-reviewers@vger.kernel.org /* new: triggers on sg_chain/sg_set_page calls */
F:   crypto/
F:   crypto/algif_*.c

/* scripts/get_maintainer.pl-aware annotation */

```

```
/* @reviewers-on-pattern sg_chain|sg_set_page net/xfrm@vger.kernel.org
 *                                           linux-mm@kvack.org
 *                                           crypto-reviewers@vger.kernel.org
 */
```

6.7 Principle: Misuse-Resistant API Design

`scatterwalk_map_and_copy(void *buf, struct scatterlist *sg, unsigned int start, unsigned int nbytes, int out)` is a textbook example of an API that allows dangerous calls because the `int out` parameter is the only distinction between safe (read) and dangerous (write) calls. The kernel community has been migrating away from this API for exactly this reason — in late 2024, Eric Biggers added `memcpy_from_sglist()` and `memcpy_to_sglist()` with distinct entry points per direction (commit series “crypto: scatterwalk — use the new scatterwalk functions”).

Vulnerable API (one entry point, mode parameter)

```
/* Caller has to remember which int means “write”. */
scatterwalk_map_and_copy(tmp + 4, dst, assoclen + cryptlen, 4, 1); /* write */
scatterwalk_map_and_copy(tmp,      src, 0,                        8, 0); /* read  */
```

Secure API (direction in the function name)

```
/* Reads are obviously different from writes at the call site. */
memcpy_from_sglist(tmp, src, /*start=*/0, /*nbytes=*/8);
memcpy_to_sglist(dst, /*start=*/assoclen + cryptlen, tmp + 4, /*nbytes=*/4);

/* memcpy_to_sglist() is the right place also to enforce a provenance check:
 * The implementation can WARN_ON if it walks into a page-cache page. */
```

Misuse-resistant APIs do not eliminate the underlying primitive; they make every dangerous call visible during code review. The grep audit for `memcpy_to_sglist` would have surfaced authencesn’s use of `dst` in roughly five seconds.

6.8 Principle: Least Exposed Surface (Don’t Expose IPsec Internals to userspace)

`authencesn` was added in 2011 as an IPsec ESP/ESN internal template. It is not a general-purpose AEAD; its scratch-on-dst pattern is excused only because the in-kernel XFRM caller controls the `dst` end-to-end. Exposing this internal template to userspace through `AF_ALG` was a foreseeable design hazard: userspace would build any caller it wanted, and the in-kernel assumption about `dst` could no longer be relied on.

A small kernel API change — a `CRYPTO_INTERNAL` flag on the template registration, and a check in `af_alg_bind()` that refuses to bind templates carrying that flag — would have made Copy Fail unexploitable while still letting IPsec use `authencesn`.

Secure rewrite (mark IPsec internals as internal)

```
/* crypto/authencesn.c – mark this template as IPsec-internal. */
static struct crypto_template crypto_authenc_esn_tmpl = {
    .name      = "authencesn",
```

```

    .create    = crypto_authenc_esn_create,
    .module    = THIS_MODULE,
    .flags     = CRYPTO_ALG_INTERNAL,      /* new */
};

/* crypto/af_alg.c - enforce at bind time. */
static int alg_bind(struct socket *sock, struct sockaddr *uaddr, int addr_len)
{
    struct crypto_alg *alg = crypto_alg_lookup(...);
    if (alg->cra_flags & CRYPTO_ALG_INTERNAL)
        return -EPERM;                      /* not exposed via AF_ALG */
    /* ... existing logic ... */
}

```

The systemic lesson — Hacker News commenter on the disclosure

“If you’re in charge of the configuration for a Linux kernel, I strongly recommend disabling all CONFIG_CRYPT_USER_API_* kconfig options. This would have made this bug, and also every past and future AF_ALG bug, unexploitable.” — Long-time Linux kernel cryptography contributor on Hacker News (2026-04-29). The least-exposed surface is the most powerful preventive control that kernel maintainers have for IPsec-internal templates.

Chapter 7 — Secure-Coding Lecture, Part 2: A Line-by-Line Secure Rewrite

Chapter 6 stated principles. This chapter shows what the two vulnerable functions look like when those principles are applied. We present the two changes side by side: vulnerable (left) versus secure (right). The secure column tracks what mainline actually shipped, with annotations marking which Chapter-6 principle each line embodies.

7.1 algif_aead._aead_recvmmsg() — The Boundary Layer

This is where Copy Fail was effectively fixed (commit a664bf3d603d, in mainline 7.0).

VULNERABLE — in-place (commit 72548b093ee3)

```
static int _aead_recvmmsg(struct sock *sk,
                        struct msghdr *msg, ...)
{
    /* setup */

    /* Copy AAD+CT into RX buffer */
    memcpy_sglist(rsgl_src, areq->tsgl,
                  used - authsize);

    /* DANGER: chain tag pages from TX SGL
     * (page-cache pages, after splice) into
     * the destination scatterlist. */
    sg_chain(rsgl_src, rsgl_count + 1,
             areq->tsgl_tag);

    /* DANGER: src == dst aliases the
     * chained page-cache pages into the
     * writable destination. */
    aead_request_set_crypt(
        &areq->cra_u.aead_req,
        rsgl_src, /* src */
        rsgl_src, /* dst */
        used, ctx->iv);

    err = crypto_aead_decrypt(
        &areq->cra_u.aead_req);
    /* authencsn writes seqno_lo here,
     * landing on a page-cache page. */
    return err;
}
```

SECURE — out-of-place (commit a664bf3d603d)

```
static int _aead_recvmmsg(struct sock *sk,
                        struct msghdr *msg, ...)
{
    /* setup */

    /* Copy only the AAD into the RX buffer.
     * Ciphertext stays in the TX SGL and is
     * read (not written) by the cipher. */
    memcpy_sglist(rsgl_dst, areq->tsgl,
                  areq->assoclen);

    /* No sg_chain into dst.
     * No page-cache page reaches dst. [§6.4] */

    /* Out-of-place: src is TX SGL (may hold
     * splice page refs but read-only here),
     * dst is the freshly allocated RX SGL. */
    aead_request_set_crypt(
        &areq->cra_u.aead_req,
        areq->tsgl, /* src */
        rsgl_dst, /* dst */
        used, ctx->iv);

    err = crypto_aead_decrypt(
        &areq->cra_u.aead_req);
    /* authencsn's scratch write at
     * dst[assoclen+cryptlen] now lands in
     * rsgl_dst — kernel-private memory. */
    return err;
}
```

Annotated reading: the secure version embodies principle §6.4 (defensive copy at the trust boundary) by structurally refusing to alias TX-side pages with the destination. It also makes principle §6.2 (ownership proof for aliasing) trivially satisfied — because there is no aliasing.

7.2 crypto_authenc_esn_decrypt() — The Scratch-Space Refactor

Mainline's fix did not touch authencsn at all. The bug was eliminated upstream. But Principle §6.3 — never use caller buffers as scratch space — suggests a defence-in-depth refactor that would have prevented this entire class of bug regardless of the algif_aead behaviour. Below is the refactor we recommend, side by side with the upstream original.

VULNERABLE — scratch on dst (in tree)

```
static int
crypto_authenc_esn_decrypt(struct aead_request
*req)
{
    /* setup, derive assoclen, cryptlen,
    * authsize, dst = req->dst */
    u8 *tmp = areq_ctx->tail;

    /* Read 8 ESN bytes from src. */
    scatterwalk_map_and_copy(
        tmp, src, 0, 8, /* out= */ 0);

    if (src == dst) {
        /* Reshuffle by WRITING dst. */
        scatterwalk_map_and_copy(
            tmp, dst, 4, 4, 1);
        /* ! Copy-Fail write target! */
        scatterwalk_map_and_copy(
            tmp + 4, dst,
            assoclen + cryptlen, 4, 1);
        dst = scatterwalk_ffwd(
            areq_ctx->dst, dst, 4);
    } else {
        /* analogous separate path */
    }

    /* compute HMAC, decrypt, then
    * decrypt_tail() restores dst. */
    return err;
}
```

SECURE — private scratch only [§6.3]

```
static int
crypto_authenc_esn_decrypt(struct aead_request
*req)
{
    struct authenc_esn_request_ctx *c =
        aead_request_ctx(req);
    u8 *seq = c->seq_scratch; /* private! */

    /* Read 8 ESN bytes from src into
    * our private buffer. NO WRITES TO DST. */
    scatterwalk_map_and_copy(
        seq, req->src, 0, 8, /* out= */ 0);

    /* Build an alternative HMAC input SGL
    * pointing at the private buffer in
    * RFC 4303 order, then the rest of src. */
    sg_init_table(c->hmac_src_sg, 3);
    sg_set_buf(&c->hmac_src_sg[0], &seq[4], 4);
    sg_set_buf(&c->hmac_src_sg[1], &seq[0], 4);
    sg_chain(c->hmac_src_sg, 3,
        scatterwalk_ffwd(
            c->ffwd_sg, req->src, 8));

    /* Feed c->hmac_src_sg to ahash, no
    * decrypt_tail() needed; dst untouched. */
    return err;
}
```

This refactor adds a 16-byte addition to the request context (seq_scratch[8] plus an 8-byte alignment slot for a small inline sg array), spends one extra sg_init_table(), and removes both the decrypt_tail() restore step and any write to dst. It implements principle §6.3 by construction: the byte rearrangement occurs entirely within the memory owned by the function. Even on a hypothetical kernel where dst contains page-cache pages, no write originates from authenc.esn.

7.3 Defence-in-Depth Layer — A Provenance-Aware Scatterwalk

Finally, regardless of whether the algif_aead or authenc.esn paths are fixed, the scatterwalk infrastructure can be hardened. This change is invisible at runtime in production builds, but it produces a WARN when CONFIG_DEBUG_CRYPTO_PROVENANCE=y.

```
/* crypto/scatterwalk.c – add to the existing file.
 * Annotated with the principle each block embodies. */

#ifdef CONFIG_DEBUG_CRYPTO_PROVENANCE
/* [§6.5] In-bounds is not the same as safe – audit page provenance
 * when a kernel crypto path writes to a scatterlist page. */
static ninline void scatterwalk_audit_write(struct scatter_walk *w)
{
    struct page *p = scatterwalk_page(w);
    struct address_space *m;
    struct inode *host;

    if (!p)
        return;
    m = READ_ONCE(p->mapping);
    if (!m)
```

```

        return;
    host = READ_ONCE(m->host);
    if (!host)
        return;
    if (S_ISREG(READ_ONCE(host->i_mode))) {
        WARN_ONCE(1,
            "scatterwalk: crypto WRITE to page-cache page "
            "of regular-file inode %lu - likely CVE-2026-31431-class bug\n",
            host->i_ino);
    }
}
#else
static inline void scatterwalk_audit_write(struct scatter_walk *w) {}
#endif

/* [§6.7] Misuse-resistant API: name distinguishes direction explicitly. */
void memcpy_to_sglist(struct scatterlist *sg, unsigned int start,
                      const void *buf, unsigned int nbytes)
{
    struct scatter_walk walk;
    struct scatterlist tmp[2];

    if (!nbytes)
        return;
    sg = scatterwalk_ffwd(tmp, sg, start);
    scatterwalk_start(&walk, sg);
    scatterwalk_audit_write(&walk);
    scatterwalk_copychunks((void *)buf, &walk, nbytes, /*out=*/1);
    scatterwalk_done(&walk, /*out=*/1, /*more=*/0);
}
EXPORT_SYMBOL_GPL(memcpy_to_sglist);

```

With these three-layered changes — out-of-place at the boundary (Chapter 7.1), private scratch in authencesn (Chapter 7.2), and provenance-aware scatterwalk (Chapter 7.3) — Copy Fail becomes a bug that requires three independent regressions to reappear. That is what defence-in-depth looks like in kernel code.

Chapter 8 — Would Fuzzing Have Caught Copy Fail?

8.1 The Honest Answer: Default Fuzzing Did Not, for Eight Years

Syzkaller has run continuously against the Linux mainline since 2015. It has had AF_ALG syscall descriptions for at least that long. It runs with KASAN, KCSAN, KMSAN, UBSAN, and lockdep enabled. Trinity, sandcastle, kAFL, Healer and several other fuzzers have also exercised the kernel. Copy Fail nonetheless lived from 2017 to 2026 — nearly nine years — without any fuzzer finding it.

There are five structural reasons. Each one is also a clue about how to do better.

8.1.1 KASAN is the wrong oracle

KASAN reports out-of-bounds, use-after-free, and uninitialised-memory accesses. Copy Fail is in-bounds; the destination scatterlist is valid, the target page is mapped, and the size is correct. KASAN sees nothing wrong. Every other memory-safety sanitiser falls into the same category: they check size and lifetime, not provenance. A bug whose effect is “the right number of bytes were written to a page that we did not realise we had access to” is below the sanitisation horizon.

8.1.2 The damaging effect is observable only in another process or syscall

After `write4()`, the corruption sits in the page cache. The crashing observable — the failed HMAC — is masked by `recvmsg()` returning `-EBADMSG`, which is a normal, expected error for a fuzzer driving random crypto inputs. The actual damage only shows up if some other process (or the same fuzzer’s next iteration) reads the file and notices its contents have changed. Standard fuzzers do not run that observability harness.

8.1.3 The bug requires a specific, rare composition

To trigger the bug a fuzzer must simultaneously: (a) open an AF_ALG socket, (b) bind it to `authencsn(hmac(sha256),cbc(aes))` specifically — not any of dozens of more common AEAD templates, (c) call `splice()` to feed the socket from a file descriptor opened on a real file, (d) issue `sendmsg(MSG_MORE)` with an AAD plus `cmsg` attributes setting the right `assoclen`, and (e) issue `recvmsg()` to actually run the decrypt. Coverage-guided fuzzers struggle to pile up that many specific conditions without structure-aware help.

8.1.4 syzkaller’s built-in AF_ALG descriptions rarely combine splice with AEAD

syzkaller has fine descriptions of AF_ALG, splice, sendmsg with cmsg, and recvmsg — but the auto-generated programs that combine them tend to mutate at the wrong granularity. By default, syzkaller treats `algif_aead’s sa_name` as a free-form string and rarely lands on `authencsn` specifically; it also tends to call splice with non-file descriptors or with the wrong file types. A targeted harness that biases toward this exact composition is necessary.

8.1.5 No oracle for cross-process state changes

Fuzzers detect bugs by observing kernel state — oopses, KASAN reports, deadlock warnings. None of them ask “did this syscall sequence change the contents of a file the caller should not have been able to write?” That is a property-based oracle, and the standard fuzzer toolchain does not provide it.

8.2 What Fuzzing Would Have Caught This

The bug is reachable; it is not exotic. The right kinds of fuzzing readily find it.

8.2.1 Property-based / metamorphic fuzzing with a file-integrity oracle

The simplest oracle is: “unprivileged user U should not be able to change the contents of any file U does not have write access to.” Encode it as a fuzz wrapper: pick a file, hash it, run a random sequence of AF_ALG syscalls, hash again, and compare. Any mismatch is a bug. This finds Copy Fail in minutes once the harness biases toward AF_ALG + splice.

8.2.2 Differential testing of in-place vs out-of-place

Most AEAD implementations expose both in-place and out-of-place modes. A differential harness submits identical inputs in both directions and verifies that the side effects are identical (file contents and scatterlist contents). Where the in-place path silently writes to a page that the out-of-place path leaves untouched, the bug surfaces.

8.2.3 Custom KASAN-like sanitiser: KPCAM (Page-Cache Access Monitor)

A purpose-built dynamic sanitiser annotates each page with a provenance tag (page-cache, anonymous, dma-coherent, kmalloc, etc.) and traps on writes whose call-site does not declare it expects that provenance. The detection logic is essentially the `scatterwalk_audit_write()` routine in Chapter 7.3, but applied across every kernel write path (`memcpy_to_page`, `copy_to_user`, etc.). This is heavier than the targeted provenance check but catches an entire class of bugs.

8.2.4 Structure-aware fuzzing with explicit AF_ALG + splice grammar

Tools like syzkaller, Healer, MoonShine, and SyzVegas can be given grammars that encode “in a single program, allocate an AF_ALG socket, bind a randomly-chosen AEAD template, splice from a file fd, then issue a `recvmsg`.” A grammar that pins authencsn finds Copy Fail in the first generation of programs.

8.2.5 Symbolic execution / static analysis assist

Symbolic execution at full kernel scale is hard, but targeted symbolic execution on `crypto_authenc_esn_decrypt()` — prompted by a simple data-flow query: “which values flow into the `dst` argument of `scatterwalk_map_and_copy` with `out=1`?” — would have flagged a write to a buffer whose pages can originate from `req->dst`, which a static caller-side audit could trace to

algif_aead's sg_chain. Klee, S2E, or even CodeQL queries (Chapter 11) close this gap without symbolic execution.

8.3 So Could It Have Been Found Earlier?

Yes. The honest assessment is: with the harnesses in Chapters 9 and 10 deployed in 2017 alongside the original commit, Copy Fail would have been detected within hours. None of those harnesses requires kernel changes. They could have lived alongside syzkaller as a small additional service.

The teaching moment

Eight years of fuzzing did not catch Copy Fail because the fuzzer's oracle was "the kernel crashed." The bug class "silent corruption of memory belonging to a different security domain" requires a different oracle: "syscall sequence by user U changed memory owned by privilege P." Building that oracle is the single highest-value extension you can make to a kernel fuzzing pipeline.

Chapter 9 — Setting Up a Kernel Fuzzing Lab to Find Copy-Fail-Class Bugs

This chapter walks through standing up a reproducible kernel fuzzing environment that would have caught Copy Fail. It is deliberately concrete: every command below is something a student can run on a workstation, and the entire lab fits on a single machine with 16 GB of RAM.

9.1 Lab Architecture

Five components:

- Host: Linux 6.x with KVM (Intel VT-x or AMD-V).
- Target kernel: a Linux mainline tree built with KASAN, KMSAN, lockdep, debug info, and the optional CONFIG_DEBUG_CRYPTO_PROVENANCE sketched in Chapter 7.
- QEMU + virtio-net for syzkaller VM lifecycle.
- syzkaller: coverage-guided fuzzer, configured with custom AF_ALG + splice + AEAD descriptions.
- A property-based oracle service that watches selected files for content changes during fuzzing.

9.2 Building a Kernel With the Right Sanitizers

The minimum configuration:

```
# Clone the target source
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux
git checkout v6.12      # pick a pre-fix version to reproduce Copy Fail

# Generate a syzkaller-friendly defconfig
make defconfig
make kvm_guest.config

# Enable sanitisers and crypto user-API surface
scripts/config \
  --enable KASAN --enable KASAN_GENERIC \
  --enable KMSAN \
  --enable KCSAN \
  --enable LOCKDEP --enable PROVE_LOCKING \
  --enable DEBUG_INFO --enable DEBUG_INFO_DWARF4 \
  --enable KCOV --enable KCOV_INSTRUMENT_ALL --enable KCOV_ENABLE_COMPARISONS \
  --enable FAULT_INJECTION --enable FAILSLAB --enable FAIL_PAGE_ALLOC \
  --enable CONFIGFS_FS --enable SECURITYFS \
  --enable CRYPTO_USER_API --enable CRYPTO_USER_API_AEAD \
  --enable CRYPTO_AUTHENCESN --enable CRYPTO_HMAC --enable CRYPTO_SHA256 \
  --enable CRYPTO_AES --enable CRYPTO_CBC

make olddefconfig
make -j$(nproc)
# Output: arch/x86/boot/bzImage and vmlinux (the latter with KASAN symbols)
```

9.3 Boot the Target in QEMU

syzkaller will manage the QEMU lifecycle itself, but it is useful to confirm the image boots:

```
# Build a minimal Debian rootfs with stress-ng, perf, gdb, python3:
syz-make-image.sh stretch ./image.img # syzkaller ships create-image.sh

qemu-system-x86_64 \
  -m 4096 -smp 2 -enable-kvm -cpu host \
  -kernel arch/x86/boot/bzImage \
  -append "root=/dev/sda console=ttyS0 nokaslr" \
  -drive file=./image.img,if=virtio \
  -nic user,model=virtio \
  -nographic
```

9.4 syzkaller Configuration

Place this at `~/syzkaller/copyfail.cfg`:

```
{
  "target": "linux/amd64",
  "http": "127.0.0.1:56741",
  "workdir": "/home/fuzz/workdir",
  "kernel_obj": "/home/fuzz/linux",
  "image": "/home/fuzz/image/stretch.img",
  "sshkey": "/home/fuzz/image/stretch.id_rsa",
  "syzkaller": "/home/fuzz/syzkaller",
  "procs": 8,
  "type": "qemu",
  "vm": {
    "count": 4,
    "kernel": "/home/fuzz/linux/arch/x86/boot/bzImage",
    "cpu": 2,
    "mem": 4096
  },
  "enable_syscalls": [
    "socket$alg", "bind$alg", "accept$alg", "setsockopt$ALG_SET_KEY",
    "setsockopt$ALG_SET_AEAD_AUTHSIZE", "sendmsg$alg", "recvmsg$alg",
    "splice", "pipe2", "openat", "read", "close", "mmap"
  ]
}
```

This restricts syzkaller to syscalls actually reachable from the bug, which is roughly an 8x speedup over default-everything. The `enable_syscalls` list is the single most useful tuning knob when hunting a specific class of bug.

9.5 The Page-Cache Integrity Oracle

syzkaller does not natively detect “syscall sequence U changed a file U does not own.” We add the oracle as a small companion service running on the host, looking inside the guest at the watched files. Implementation in 80 lines of Python, executed inside the guest as a background process:

```
#!/usr/bin/env python3
```

```
# /usr/local/bin/pc_integrity_oracle.py - runs as root inside the guest
import hashlib, os, time

WATCHED = ["/etc/passwd", "/usr/bin/su", "/etc/group"]
INTERVAL = 0.25 # seconds

def fhash(path):
    try:
        with open(path, "rb") as f:
            return hashlib.sha256(f.read()).hexdigest()
    except FileNotFoundError:
        return None

baseline = {p: fhash(p) for p in WATCHED}
print("baseline", baseline, flush=True)

while True:
    time.sleep(INTERVAL)
    # Flush page cache only on disk side; we want to see the cached view too.
    cur = {p: fhash(p) for p in WATCHED}
    for p in WATCHED:
        if cur[p] != baseline[p]:
            # PANIC the kernel so syzkaller picks it up as a crash.
            with open("/proc/sysrq-trigger", "w") as s:
                s.write("c") # crash, captured by syzkaller
            # Unreachable.
            os._exit(1)
```

The trick: syzkaller treats any kernel oops as a crash to report and triage. By turning a file-integrity mismatch into a deliberate kernel panic, we coerce syzkaller's existing infrastructure into reporting our custom invariant. A more elegant variant exposes a syscall (or a debugfs entry) that the fuzzer polls, allowing syzkaller to treat its return value as an oracle, but the panic-on-violation approach works out of the box.

9.6 Iteration and Triage

With everything wired up, start the campaign:

```
syz-manager -config copyfail.cfg
```

Visit <http://127.0.0.1:56741> for the web UI. On a kernel built without the fix, the page-cache oracle fires within minutes. The crash is reproducible. The reproducer is a short C program: open AF_ALG, bind authencesn, splice from /etc/passwd, sendmsg+recvmsg — i.e., the exact Theori PoC, reconstructed from scratch by the fuzzer.

Chapter 10 — Three Reusable Harnesses

Building on the lab in Chapter 9, this chapter provides three concrete artefacts: a syzkaller pseudo-syscall description, a Python property-based harness, and an eBPF runtime invariant detector. Each can be used independently.

10.1 Harness A — syzkaller AF_ALG + splice Pseudo-Syscalls

Drop this file at `sys/linux/copyfail.txt` in the syzkaller source tree and rebuild. It biases generation toward the exact composition that triggers Copy Fail.

```
# sys/linux/copyfail.txt - syzlang descriptions for Copy-Fail-class fuzzing.
include <linux/socket.h>
include <linux/if_alg.h>
include <uapi/asm-generic/fcntl.h>

# Pin AF_ALG bind() onto the small set of AEAD templates we want to exercise.
resource sock_alg_aead[sock]
resource alg_op_fd[fd]

socket$alg_aead(domain const[AF_ALG], type const[SOCK_SEQPACKET], proto const[0])
sock_alg_aead

bind$alg_authencesn(fd sock_alg_aead, addr ptr[in, sockaddr_alg_authencesn], len
bytesize[addr])

sockaddr_alg_authencesn {
    family    const[AF_ALG, int16]
    type      stringnoz["aead\x00", 14]
    feat      const[0, int32]
    mask      const[0, int32]
    name      stringnoz["authencesn(hmac(sha256),cbc(aes))\x00", 64]
}

setsockopt$ALG_SET_KEY_aead(fd sock_alg_aead, level const[SOL_ALG],
                             opt const[ALG_SET_KEY], val ptr[in, array[int8, 40]],
                             len const[40])

setsockopt$ALG_SET_AEAD_AUTHSIZE(fd sock_alg_aead, level const[SOL_ALG],
                                  opt const[ALG_SET_AEAD_AUTHSIZE], val const[0],
                                  len intptr[4:32])

accept$alg_op(fd sock_alg_aead, addr const[0], len const[0]) alg_op_fd

# AAD followed by attribute-cmsg's. The msg_iov and msg_control are the
# interesting variation surface for the fuzzer; len is constrained to the
# fast path that exercises the in-place SGL chaining.
sendmsg$alg_aad_more(fd alg_op_fd, msg ptr[in, msghdr_alg_aad_more], flags const[MSG_MORE])

msghdr_alg_aad_more {
    msg_name      const[0, intptr]
    msg_namelen   const[0, int32]
    msg_iov       ptr[in, iovec_aad]
    msg_iovlen    const[1, intptr]
    msg_control    ptr[in, array[cmsg_alg, 3]]
    msg_ctrllen   bytesize[msg_control, intptr]
    msg_flags     const[0, int32]
```

```

}

iovec_aad {
    iov_base ptr[in, array[int8, 8]]    # the 8-byte AAD
    iov_len  const[8, intptr]
}

cmsg_alg [
    op      cmsg_alg_set_op
    iv      cmsg_alg_set_iv
    assoc   cmsg_alg_set_assoclen
]

cmsg_alg_set_op {
    cmsg_len const[20, intptr]    # CMSG_LEN(4) on amd64
    cmsg_level const[SOL_ALG, int32]
    cmsg_type const[ALG_SET_OP, int32]
    val      const[ALG_OP_DECRYPT, int32]    # bias to decrypt
}

cmsg_alg_set_iv {
    cmsg_len const[36, intptr]    # CMSG_LEN(20) (4-byte len + 16-byte IV)
    cmsg_level const[SOL_ALG, int32]
    cmsg_type const[ALG_SET_IV, int32]
    ivlen    const[16, int32]
    iv       array[int8, 16]
}

cmsg_alg_set_assoclen {
    cmsg_len const[20, intptr]
    cmsg_level const[SOL_ALG, int32]
    cmsg_type const[ALG_SET_AEAD_ASSOCLEN, int32]
    val      const[8, int32]    # match the bug's typical assoclen
}

# Splice from a real file fd into a pipe, then from the pipe into the alg fd.
splice$file_to_pipe(fd_in fd_regular, off_in ptr[in, fileoff, opt],
    fd_out fd_wpipe, off_out const[0],
    len intptr[4:65536], flags const[0])
splice$pipe_to_alg(fd_in fd_rpipe, off_in const[0],
    fd_out alg_op_fd, off_out const[0],
    len intptr[4:65536], flags const[0])

recvmsg$alg(fd alg_op_fd, msg ptr[in, msghdr], flags const[0])

```

In a syzkaller run with the page-cache oracle from §9.5 enabled, this description shifts the prior over syscall combinations sufficiently for Copy Fail within the first hour. Without the oracle, the bug remains invisible — because syzkaller still has no reason to flag the -EBADMSG return as interesting.

10.2 Harness B — Python Property-Based Harness with Hypothesis

This harness runs in userspace, requires no kernel modifications, and demonstrates the property-based approach end-to-end. It works on any vulnerable kernel and can be added to a regression test suite.

```
#!/usr/bin/env python3
# pc_integrity_fuzz.py — unprivileged AF_ALG fuzzer with file-integrity oracle.
# Requires Python 3.10+ and the 'hypothesis' package.
import hashlib, os, socket, struct, subprocess, tempfile, hypothesis
from hypothesis import given, strategies as st, settings

WATCHED = [] # filled in main()

def hashes():
    return {p: hashlib.sha256(open(p, 'rb').read()).hexdigest() for p in WATCHED}

def af_alg_sequence(aad, assoclen, splice_offset, splice_len, target_fd):
    """Issue a sendmsg+splice+recvmsg sequence against algif_aead."""
    SOL_ALG, MSG_MORE = 279, 32768
    ALG_SET_KEY, ALG_SET_IV = 1, 2
    ALG_SET_OP, ALG_SET_ASSOC, ALG_SET_AUTH = 3, 4, 5

    s = socket.socket(38, socket.SOCK_SEQPACKET, 0) # AF_ALG = 38
    s.bind(("aead", "authencsn(hmac(sha256),cbc(aes))"))
    s.setsockopt(SOL_ALG, ALG_SET_KEY,
                 bytes.fromhex('0800010000000010' + '00' * 32))
    s.setsockopt(SOL_ALG, ALG_SET_AUTH, None, 4)
    op, _ = s.accept()
    iv = b'\x10' + b'\x00' * 15
    op.sendmsg(
        [aad],
        [
            (SOL_ALG, ALG_SET_IV, struct.pack("I", 16) + iv),
            (SOL_ALG, ALG_SET_OP, struct.pack("I", 0)), # decrypt
            (SOL_ALG, ALG_SET_ASSOC, struct.pack("I", assoclen)),
        ],
        MSG_MORE,
    )
    r, w = os.pipe()
    os.splice(target_fd, w, splice_len, offset_src=splice_offset)
    os.splice(r, op.fileno(), splice_len)
    try: op.recv(8 + splice_len)
    except OSError: pass
    os.close(r); os.close(w); op.close(); s.close()

@given(
    aad = st.binary(min_size=8, max_size=8),
    assoclen = st.integers(min_value=4, max_value=64).filter(lambda x: x % 4 == 0),
    so = st.integers(min_value=0, max_value=512),
    sl = st.integers(min_value=4, max_value=128).filter(lambda x: x % 4 == 0),
)
@settings(max_examples=10000, deadline=None)
def test_no_modification(aad, assoclen, so, sl):
    """Invariant: unprivileged AF_ALG ops must not change watched files."""
    before = hashes()
    for path in WATCHED:
        fd = os.open(path, os.O_RDONLY)
        try: af_alg_sequence(aad, assoclen, so, sl, fd)
        finally: os.close(fd)
```

```

after = hashes()
if before != after:
    raise AssertionError(
        f"PAGE CACHE CORRUPTION! aad={aad.hex()} assoclen={assoclen} "
        f"so={so} sl={sl}; diff={set(before.items())^set(after.items())}")

if __name__ == "__main__":
    # Use temp test files only; never touch real /etc/passwd in a fuzz run.
    for c in ("alice:x:1000:1000::/home/alice:/bin/bash\n" * 4,
              b'\x7fELF' + b'A' * 1024):
        f = tempfile.NamedTemporaryFile(delete=False)
        f.write(c.encode() if isinstance(c, str) else c)
        f.flush()
        WATCHED.append(f.name)
    test_no_modification()

```

Run it

On a vulnerable kernel: `python3 pc_integrity_fuzz.py`. The harness fires `AssertionError` within a few hundred iterations — typically in under 30 seconds on a developer laptop. The reproducer is the exact AAD, assoclen, offset, and length that triggered the corruption. On a patched kernel, the harness runs to completion (10,000 examples) without finding a violation.

10.3 Harness C — eBPF Runtime Invariant Detector

The last harness is meant for production-style deployment, not fuzzing. It attaches a kprobe to `scatterwalk_map_and_copy()` (and the newer `memcpy_to_sglist()`) and alerts whenever a write would land on a page-cache page of a regular file. It is the runtime cousin of Chapter 7.3's in-tree provenance check and can be deployed without rebuilding the kernel.

```

// SPDX-License-Identifier: GPL-2.0
// pc_write_detector.bpf.c — build with libbpf + CO-RE.
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

char LICENSE[] SEC("license") = "GPL";

struct event {
    __u64 pid_tgid;
    __u64 inode;
    char comm[16];
};

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 20);
} events SEC(".maps");

static __always_inline int handle(struct scatterlist *sg)
{
    if (!sg) return 0;
    unsigned long pl = BPF_CORE_READ(sg, page_link);
    struct page *p = (struct page *) (pl & ~3UL);
    if (!p) return 0;

```

```

    struct address_space *m = BPF_CORE_READ(p, mapping);
    if (!m) return 0;
    struct inode *host      = BPF_CORE_READ(m, host);
    if (!host) return 0;

    umode_t mode = BPF_CORE_READ(host, i_mode);
    if ((mode & 00170000) != 0100000) /* not S_IFREG */
        return 0;

    struct event *e = bpf_ringbuf_reserve(&events, sizeof(*e), 0);
    if (!e) return 0;
    e->pid_tgid = bpf_get_current_pid_tgid();
    e->inode     = BPF_CORE_READ(host, i_ino);
    bpf_get_current_comm(&e->comm, sizeof(e->comm));
    bpf_ringbuf_submit(e, 0);
    return 0;
}

// scatterwalk_map_and_copy(buf, sg, start, nbytes, out)
SEC("kprobe/scatterwalk_map_and_copy")
int BPF_KPROBE(scatterwalk_write_probe,
               void *buf, struct scatterlist *sg,
               unsigned int start, unsigned int nbytes, int out)
{
    if (out) handle(sg);
    return 0;
}

// memcpy_to_sglst(sg, start, buf, nbytes) — newer mainline API
SEC("kprobe/memcpy_to_sglst")
int BPF_KPROBE(memcpy_to_sglst_probe,
               struct scatterlist *sg,
               unsigned int start, const void *buf, unsigned int nbytes)
{
    handle(sg);
    return 0;
}

```

Build with libbpf, load with bpftool or a small loader, and consume the ring buffer from userspace. Each event is the (pid, comm, inode) tuple of a kernel write that landed on a regular-file page-cache page. In a healthy system, the stream is empty. On a vulnerable kernel under Copy Fail exploitation, every write4() call produces an event with comm="python3" (or the attacker process) and inode = /etc/passwd or /usr/bin/su.

Production deployment

This BPF program is the same detection logic Sysdig, ReversingLabs, and Falco use for the AF_ALG-via-authencesn pattern, generalised one layer down. Attaching to scatterwalk_map_and_copy gives you coverage of every crypto write through that helper, not just AEAD. The cost is a kprobe on a hot path — measure overhead on your specific workload before fleet rollout. Cloudflare's bpf-lsm approach (deny socket_bind for AF_ALG except for allow-listed processes) is a complementary preventive control.

Chapter 11 — What Code Review Should Have Caught

Fuzzing finds bugs at runtime. Static analysis is the cheaper cousin — it finds them during code review, before they ever ship. Two rule families would have flagged Copy Fail in the 2017 commit. Both are pattern-based, and both can be implemented today.

11.1 CodeQL Rule: In-Place AEAD with Externally Chained Pages

The rule below traverses the kernel's call graph looking for: a call to `sg_chain()` that takes its second argument from a function parameter or external SGL, followed in the same function by an `aead_request_set_crypt()` call whose `src` and `dst` arguments are the same SGL produced from the chaining. It is a textbook taint-tracking query.

```
/**
 * @name In-place AEAD on a scatterlist chained from external pages
 * @description Flags algif_aead / dm_crypt-style callers that submit an AEAD
 *               request in-place (src == dst) on a destination scatterlist that
 *               was extended via sg_chain() with pages from outside the function.
 *               This is the structural shape of CVE-2026-31431.
 * @kind problem
 * @id cpp/in-place-aead-chained
 * @severity error
 * @precision medium
 * @tags security crypto kernel
 */
import cpp
import semmle.code.cpp.dataflow.TaintTracking

class SgChainCall extends FunctionCall {
  SgChainCall() { this.getTarget().hasName("sg_chain") }
  Expr getResultSgl() { result = this.getArgument(0) }
  Expr getChainedFrom() { result = this.getArgument(2) }
}

class AeadSetCrypt extends FunctionCall {
  AeadSetCrypt() { this.getTarget().hasName("aead_request_set_crypt") }
  Expr getSrc() { result = this.getArgument(1) }
  Expr getDst() { result = this.getArgument(2) }
}

from SgChainCall chain, AeadSetCrypt set, Function f
where
  chain.getEnclosingFunction() = f and
  set.getEnclosingFunction() = f and
  // Taint: the chained-into SGL is the destination of an in-place AEAD
  TaintTracking::localTaint(
    DataFlow::exprNode(chain.getResultSgl()),
    DataFlow::exprNode(set.getDst())) and
  // In-place semantics: src and dst are the same SGL expression
  set.getSrc().toString() = set.getDst().toString()
select set,
  "In-place AEAD on a destination scatterlist that was extended at $@" +
  + "with pages from " + chain.getChainedFrom().toString()
  + ". This is the CVE-2026-31431 shape – if any of those pages can be "
  + "page-cache pages from splice(), an authenc:sn-class write will corrupt them.",
  chain, chain.toString()
```

Running this query against the 2017 algif_aead.c diff returns exactly one alert — the offending sg_chain followed by the in-place aead_request_set_crypt. The cost of running this CodeQL pack on every kernel-patch series is on the order of ten minutes; the cost of missing the alert was eight years of exposure.

11.2 Smatch/Sparse Rule: Scratch Write into the AEAD Destination

Smatch and sparse are lighter-weight, kernel-resident static analysers. A custom Smatch check can flag any scatterwalk_map_and_copy() (or memcpy_to_sglist()) call whose dst argument is req->dst and whose start argument is greater than or equal to req->assoclen + req->cryptlen — i.e., a write past the legitimate output region of an AEAD destination.

```
/* check_aead_scratch_write.c – Smatch check */
static void match_scatterwalk_write(struct expression *call)
{
    struct expression *dst    = get_argument_from_call_expr(call->args, 1);
    struct expression *start  = get_argument_from_call_expr(call->args, 2);
    struct expression *out    = get_argument_from_call_expr(call->args, 4);
    sval_t out_val;

    if (!get_value(out, &out_val) || out_val.value != 1)
        return; /* only writes interest us */

    /* dst is req->dst? */
    if (!expr_is_aead_request_dst(dst))
        return;

    /* start >= assoclen + cryptlen? */
    if (start_is_at_or_after_aead_tag_region(start))
        sm_warning("AEAD scratch write at dst tag offset – review "
                  "for CVE-2026-31431-class scenarios");
}

void check_aead_scratch_write(int id)
{
    add_function_hook("scatterwalk_map_and_copy",
                     &match_scatterwalk_write, NULL);
    add_function_hook("memcpy_to_sglist",
                     &match_scatterwalk_write, NULL);
}
```

Smatch alerts on crypto/authencesn.c the moment this check is enabled. The maintainer can then make a judgment: in 2011, the answer was “safe — only XFRM calls this, dst is private.” In 2017, the right answer changed; the check would have made it visible exactly when a reviewer needed to ask the right question.

11.3 The Limitations of Static Analysis

Neither rule above is a guaranteed catch. They are precise (low false-positive) but require a maintainer to write them. The Smatch check above did not exist in 2017 because no one had a reason to write it. This is the bootstrap problem for static analysis: rules are derived from known bugs. CVE-2026-31431 is now the bug that justifies adding both rules to the kernel's standard CI matrix — turning it from a lesson into a permanent guardrail.

Chapter 12 — Operational Mitigations for Defenders

This chapter restates the mitigation guidance from a defender's perspective, prioritising what works and explaining what does not.

12.1 Patch the Kernel

Mainline 7.0 (a664bf3d603d), stable 6.19.12 (ce42ee423e58), and 6.18.22 (fafa0fa2995a) contain the fix. All major distributions (Ubuntu, RHEL, SUSE, Debian, Amazon Linux, Arch, Fedora, Rocky/AlmaLinux, Oracle, CloudLinux) have shipped patched kernels. Patching and rebooting are the only complete fixes. A common operational mistake is installing a fixed package without rebooting; verify with `uname -r` post-deployment.

12.2 If You Cannot Reboot Immediately, Block AF_ALG

Two complementary options:

- Turn off the `algif_aead` module load:

```
echo "install algif_aead /bin/false" > /etc/modprobe.d/copy-fail.conf
rmmod algif_aead 2>/dev/null || true
# For built-in (non-modular) kernels, use kernel command-line instead:
# add initcall_blacklist=algif_aead_init to GRUB_CMDLINE_LINUX
```

- Block `socket(AF_ALG, ...)` via `seccomp` at container or `systemd-unit` level. Cloudflare's `bpf-lsm` hooks `socket_bind` on `AF_ALG` and denies the call to all but allow-listed binaries; this is the surgical option when an internal service depends on `AF_ALG`.

None of `dm-crypt/LUKS`, `kTLS`, `IPsec/XFRM`, `OpenSSL`, `GnuTLS`, or `SSH` routes through `AF_ALG`; disabling `algif_aead` is safe for the overwhelming majority of workloads. `cryptsetup`, `bluez`, `iwd`, `libkcapi`, and `OpenSSL's` `afalg` engine are the legitimate consumers — audit your fleet with `prometheus-ebpf-exporter` before fleet-wide enforcement.

12.3 Containers and Kubernetes

Do not assume `RuntimeDefault` `seccomp`, `Pod Security Standards Restricted`, or dropping all capabilities prevents Copy Fail. Independent reproductions on `Talos/containerd` and `EKS/containerd` confirm that `PSS-Restricted` pods can open `AF_ALG` sockets and reach the vulnerable path. Apply a `Localhost` `seccomp` profile denying `socket()` when the first argument equals 38 (`AF_ALG`).

12.4 Detection

- The Falco /Sysdig rule “AF_ALG Page Cache Poisoning Leading to Privilege Escalation” matches non-root processes that create AF_ALG sockets.
- ReversingLabs YARA rules anchor on the immutable string `authencesn(hmac(sha256),cbc(aes))` — a functional constraint of the exploit.
- The eBPF detector from Chapter 10.3 detects the underlying write primitive regardless of the user-mode wrapper.
- Audit rules for `socket(AF_ALG,...) + sendmsg(MSG_MORE) + splice(pipe → AF_ALG)` sequences combined with `execve()` of `setuid` binaries.

12.5 Things That Do Not Help

- `chmod 4711` of `setuid` binaries — the `/etc/passwd` path bypasses this entirely (Chapter 5.5).
- On-disk integrity tools (Tripwire, AIDE, IMA file hashes, container image scanners) — the on-disk file is never modified.
- AppArmor / SELinux in default “unconfined” mode — they help only when policies explicitly restrict AF_ALG to specific services.
- “Container escape resistance” claims are based on capability drops or PSS — the bug crosses container boundaries via the host page cache.

Chapter 13 — Lessons Learned and Takeaways

Copy Fail is, more than anything, an instructive bug. Eight non-obvious points to internalise:

1. **Composition is the new attack surface.** The 2011, 2015, and 2017 commits were each individually correct. The bug is in their composition, and no individual reviewer was responsible for it.
2. **Memory safety $\neq$ security.** KASAN, sanitisers, fuzzers, and even Rust's borrow checker would not have caught this without provenance information. Provenance is a first-class safety property.
3. **Never use caller buffers as scratch space.** authencesn's 8 bytes of micro-optimisation cost eight years of unprivileged page-cache writes. Allocate a private scratch buffer.
4. **Trust boundaries must copy, not chain.** When pages flow across a trust boundary (splice() into AF_ALG), defensive copy is the safe default. Reverting to out-of-place was the fix.
5. **Misuse-resistant APIs matter.** scatterwalk_map_and_copy(..., out=1) buried the dangerous mode in an int. memcpy_to_sglst() puts it in the name. Same primitive; vastly different code review.
6. **The least exposed surface beats clever fixes.** Marking authencesn as CRYPTO_ALG_INTERNAL and refusing to bind internal templates via AF_ALG would have prevented Copy Fail and every comparable future bug at the cost of zero performance.
7. **Fuzzing is only as good as its oracle.** Eight years of syzkaller missed this because the oracle was "the kernel crashed." Property-based oracles ("file contents must not change under unprivileged access") catch entire bug classes that kernel-crash oracles cannot.
8. **AI-assisted research is not the threat — it is the timeline accelerator.** Xint Code surfaced this bug in roughly one hour of automated scanning, with one operator prompt. Defenders should anticipate an elevated rate of similar discoveries against any complex, user-callable kernel surface. The right response is to build the harnesses in Chapters 9–10 into your own CI.

SUMMARY: The lecture in one paragraph

Copy Fail teaches us that secure systems coding is about invariants, not just bounds. Make ownership and provenance explicit; never alias buffers across trust domains without a proof; never use caller buffers as scratch; expose only what userspace must see; and write your fuzz oracles to check the property you actually care about. The bug existed for eight years because the kernel community optimised for size and lifetime safety. The next class of bugs will live in the same shadow until we extend our tools to cover provenance.

Chapter 14 — References

- Copy Fail landing page (Theori). <https://copy.fail/>
- r3tr074, “CVE-2026-31431: Copy Fail” — the source blog this lecture re-organises and extends. <https://retr0.zip/blog/cve-2026-31431-copy-fail.html>
- Theori / Xint Code, “Copy Fail: 732 Bytes to Root on Every Major Linux Distribution.” — primary disclosure and root-cause analysis. <https://xint.io/blog/copy-fail-linux-distributions>
- Theori PoC (CVE-2026-31431). <https://github.com/theori-io/copy-fail-CVE-2026-31431>
- Kernel commit 72548b093ee3 (2017) — in-place AEAD optimisation that introduced the bug. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=72548b093ee38a6d4f2a19e6ef1948ae05c181f7>
- Kernel commit a664bf3d603d (2026) — the fix; reverts to out-of-place. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=a664bf3d603dc3bdcf9ae47cc21e0daec706d7a5>
- Stable backports: ce42ee423e58 (6.19.12), fafe0fa2995a (6.18.22). <https://git.kernel.org/stable/c/afe0fa2995a0f7073c1c358d7d3145bcc9aedd8>
- Openwall oss-security disclosure thread, April 29 2026. <https://www.openwall.com/lists/oss-security/2026/04/29/23>
- Linux kernel crypto userspace interface documentation. <https://docs.kernel.org/crypto/api-intro.html>
- Dirty COW (CVE-2016-5195) — page-cache COW race. <https://dirtycow.ninja/>
- Cloudflare Engineering blog, “How Cloudflare responded to the Copy Fail Linux vulnerability” (bpf-lsm mitigation). <https://blog.cloudflare.com/copy-fail-linux-vulnerability-mitigation/>
- Sysdig Threat Research Team, CVE-2026-31431 analysis and Falco rule. <https://www.sysdig.com/blog/cve-2026-31431-copy-fail-linux-kernel-flaw-lets-local-users-gain-root-in-seconds>
- Juliet Security, “Copy Fail in Kubernetes: RuntimeDefault Did Not Block AF_ALG.” <https://juliet.sh/blog/we-tested-copy-fail-in-kubernetes-pss-restricted-runtime-default-af-alg>
- ReversingLabs, “Copy Fail Flaw: 5 YARA Rules.” <https://www.reversinglabs.com/blog/copy-fail-5-yara-rules>
- AlmaLinux advisory and patched kernels, 2026-05-01. <https://almalinux.org/blog/2026-05-01-cve-2026-31431-copy-fail/>
- GitHub Advisory Database — GHSA-2274-3hgr-wxv6. <https://github.com/advisories/GHSA-2274-3hgr-wxv6>
- tgies/copy-fail-c — cross-platform C port of the PoC. <https://github.com/tgies/copy-fail-c>
- rootsecdev/cve_2026_31431 — /etc/passwd exploitation PoC. https://github.com/rootsecdev/cve_2026_31431
- V4bel/dirtyfrag — follow-on bug (Dirty Frag) using the same authencsn pattern in xfrm/esp. <https://github.com/V4bel/dirtyfrag>
- syzkaller documentation — syzlang syntax and kernel fuzzing setup. https://github.com/google/syzkaller/blob/master/docs/syscall_descriptions_syntax.md

— The End —